

Chapter 2

Database System Concepts and Architecture

1

1

Outline

- DB Schemas, States (aka Instances)
- Three-Schema Architecture
- Data Independence
- DBMS Languages and Interfaces
- Centralized and Client-Server Architectures
- Classification of DBMSs

2

2

DB Schema vs. State (aka Instance)

- **Database Schema:**
 - The **description** of a database.
 - Includes the descriptions of **database structure**, **data types**, and **constraints** on the database.
- **Database Schema Diagram**
 - An illustrative display of a database schema.
- **Database State:**
 - Refers to the **content** of a database at a particular moment. This includes the collection of all the data in the database.
 - Also called **database instance** (or occurrence).
 - A DB state is **valid** if it satisfies the structure and constraints of the database schema.

3

3

Example of a DB Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

Figure 2.1 Schema diagram for the database in Figure 1.2.

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

4

4

Example of a DB State

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores
student and course
information.

5

5

Three-Schema Architecture

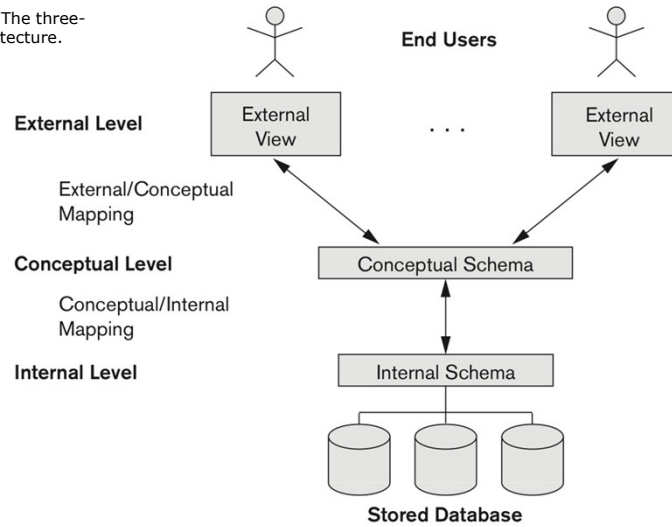
- Defines DB schemas at three levels:
 - The **Internal schema** describes the physical storage structures and access paths (e.g., indexes).
 - The **Conceptual schema** describes the logical structure and constraints for the whole database.
 - The **External schema** describes the DB view of a user group.
- Proposed to support:
 - **Program-data independence.**
 - **Multiple views** of the data.

6

6

Three-Schema Architecture

Figure 2.2 The three-schema architecture.



7

Example Three Schemas of a DB

- External schemas

User1's View

EMP#	Salary
------	--------

User2's View

EMP#	Dept#
------	-------

- Conceptual Schema

EMP

EMP#	Salary	Dept#
------	--------	-------

- Internal Schema

Stored-EMP	Length = 18 bytes
prefix	6 bytes
EMP#	6 bytes
Dept#	4 bytes
Salary	2 bytes

8

8

Three-Schema Architecture (cont'd)

- Mappings between schema levels are needed to transform user requests and retrieved data.
 - Each user group may refer to its own external schema. Hence, the DBMS transforms a request specified on an external schema into a request on the conceptual schema, then into a request on the internal schema for execution.
 - Data extracted from the stored DB is reformatted to match the user's external view (e.g., renaming the attributes in the result of an SQL query for display).

9

9

Data Independence

- **Logical Data Independence:**
 - The capacity to change the conceptual schema without having to change the external schemas and their associated application programs.
- **Physical Data Independence:**
 - The capacity to change the internal schema without having to change the conceptual schema.
 - For example, the internal schema may be changed when certain file organizations are changed or new indexes are created to improve database performance.

10

10

Data Independence (cont'd)

- When a schema at a lower level is changed, only the mappings between this schema and higher-level schemas need to be changed by the DBMS.
- The higher-level schemas themselves are not changed. Hence, the application programs need not be changed since they refer to the external schemas.

11

11

DBMS Languages

- **Data Definition Language (DDL):**
 - Used by the DBA and database designers to specify the conceptual schema of a database.
 - In many DBMSs, the DDL is also used to define internal and external schemas (i.e., views). For example,

```
CREATE INDEX Name_index
ON STUDENT(Name);
```
 - In some DBMSs, separate **Storage Definition Language (SDL)** is used to define the internal schema.
 - SDL is typically realized via DBMS commands provided to the DBA and database designers.

12

12

DBMS Languages (cont'd)

- **Data Manipulation Language (DML)** is used for retrieval, insertion, deletion, and modification of the data.
 - High-level (or Non-procedural) DML
 - is **declarative**: specify what data to retrieve rather than how to retrieve it.
 - specifies and retrieves many records by a single DML statement.
 - A high-level DML used in a stand-alone, interactive manner is called a **query language**, (e.g., SQL (Structured Query Language)).
 - Low-level (or Procedural) DML
 - Typically retrieves individual records from the DB.
 - must be embedded in a program written in a general-purpose programming language (which is called a **host language**).

13

13

Comparison between a Query Language and a Low-level DML

EMPLOYEE

<u>SSN</u>	FNAME	MINIT	LNAME	ADDRESS	SALARY
------------	-------	-------	-------	---------	--------

Query: Retrieve the names of the employees whose salary is less than \$50,000.

In SQL:

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE SALARY < 50000;
```

In Hierarchical DML:

```
$GET FIRST EMPLOYEE WHERE SALARY < '50000.00';
while DB_STATUS = 0 do
begin
  writeln (P_EMPLOYEE.FNAME, P_EMPLOYEE.LNAME);
  $GET NEXT EMPLOYEE WHERE SALARY < '50000.00'
end;
```

14

14

Example of Embedded SQL Statement

Figure 10.2 Program segment E1, a C program segment with embedded SQL.

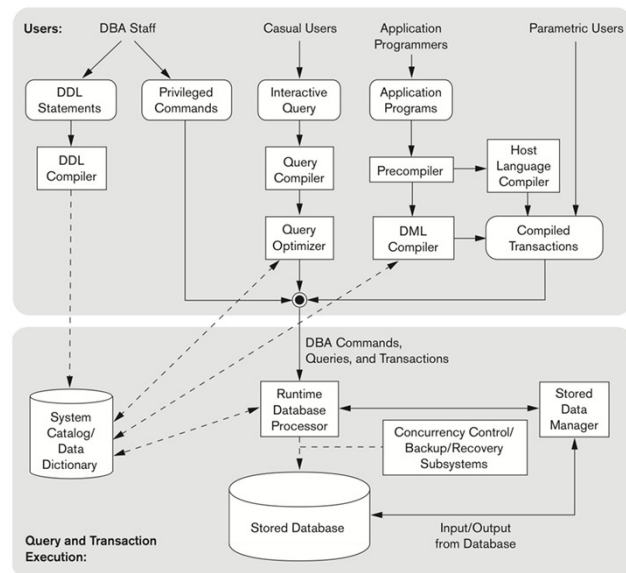
```
//Program Segment E1:
0) loop = 1 ;
1) while (loop) {
2)   prompt("Enter a Social Security Number: ", ssn) ;
3)   EXEC SQL
4)     SELECT Fname, Minit, Lname, Address, Salary
5)     INTO :fname, :minit, :lname, :address, :salary
6)     FROM EMPLOYEE WHERE Ssn = :ssn ;
7)   if (SQLCODE == 0) printf(fname, minit, lname, address, salary)
8)   else printf("Social Security Number does not exist: ", ssn) ;
9)   prompt("More Social Security Numbers (enter 1 for Yes, 0 for No): ", loop) ;
10) }
```

15

15

Typical DBMS Component Modules

Figure 2.3
Component modules
of a DBMS and their
interactions.



16

16

Typical DBMS Component Modules (cont'd)

- The **DDL compiler** processes the schema definitions specified in DDL, and stores the descriptions of schemas in the DB system catalog.
- The **Query compiler** compiles the query into an internal form (such as a query tree); and the **Query optimizer** generates an execution plan by selecting the sequence of operations and their operation algorithms.
- The **Precompiler** extracts DML commands from the application program. These commands are compiled into object codes by the **DML compiler**.
- The **Runtime DB Processor** executes (1) privileged commands, (2) executable query plans, and (3) compiled transactions.
- The **Stored Data Manager** module controls the access to the data and metadata stored on disk.

17

17

Centralized and Client-Server DB Systems

- Centralized DB system:
 - Combines everything into a single system including DBMS software, databases, hardware, application programs, and user interface programs.
 - A DB user can still connect through a remote terminal — however, all processing is done at a centralized site.

18

18

A Physical Centralized Architecture

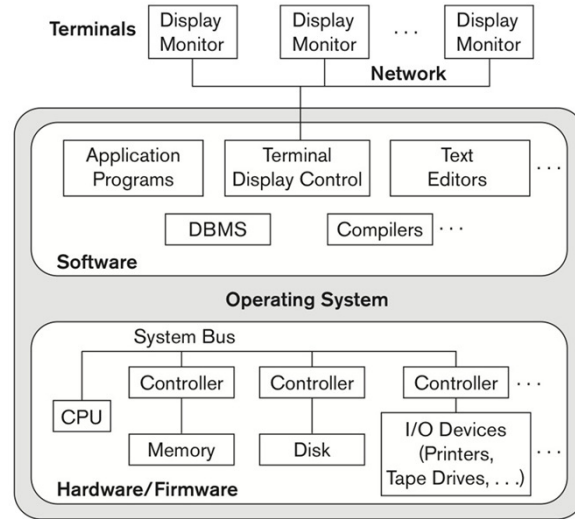


Figure 2.4 A physical centralized architecture.

19

19

Two-tier Client/Server Architecture

Figure 2.5
Logical two-tier
client/server
architecture.

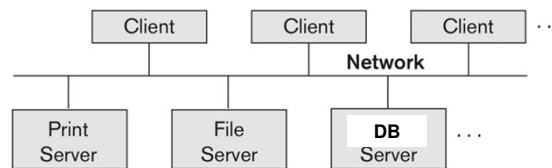
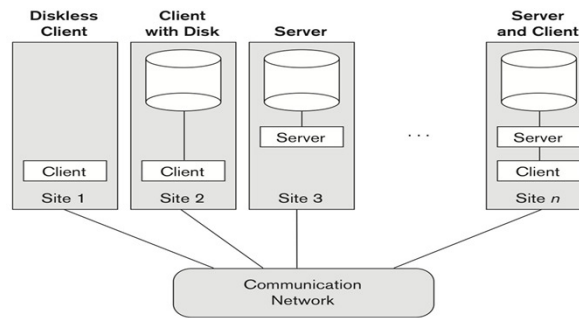


Figure 2.6
Physical two-tier
client/server
architecture.



20

20

Clients

- The user interface programs and application programs can run on the client side.
- Provide appropriate interfaces through a client software module to access and utilize the various server resources.
- Clients may be diskless machines or PCs or workstations with the client software installed.
- Connected to the servers via some form of a network.
 - local area network (LAN), wireless network, etc.

21

21

Database Server

- Provides database query and transaction services to the clients.
- Relational DB servers are often called SQL servers, query servers, or transaction servers.
- Applications running on clients utilize an Application Programming Interface (API) to access databases on the DB servers via standard software interface such as:
 - ODBC (Open Database Connectivity)
 - JDBC (Java Database Connectivity)
- Client and server must have installed appropriate client module and server module software for ODBC or JDBC.
 - See Chapter 10 (in 7th edition) for more details.

22

22

Three-Tier Architecture

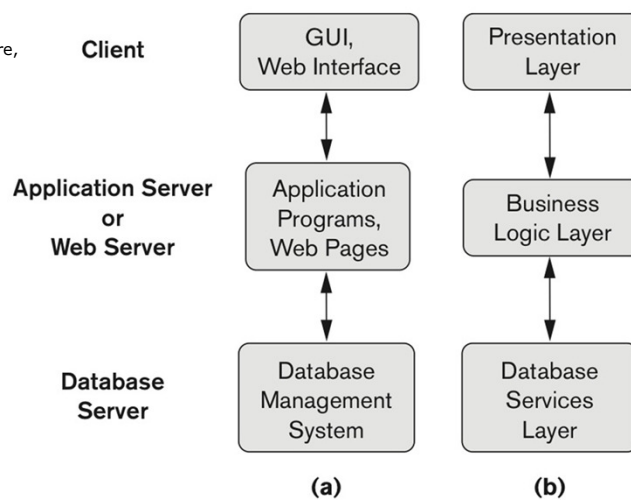
- *Layering* deals with the vertical organization of services into layers of abstraction. *Tiering* is a technique to organize and place the functionality of a given layer into appropriate servers and physical nodes.
- Three-tier architecture uses intermediate layer (or middle tier), called *Application Server* or *Web Server*, between the client and the DB server:
 - Running application programs and storing business rules (procedures or constraints) that are used to access the corresponding data from the database server.
 - Database server is accessible only via middle tier; that means, clients cannot directly access database server.
 - Can improve the database security by checking a client's credentials before forwarding a request to the database server.
 - We don't need to install/update the application programs on many client machines.
- Common for Web applications.

23

23

Three-tier client-server architecture

Figure 2.7 Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.



24

24

Classification of DBMSs

- Based on the data model used:
 - **Traditional:** Relational, Network, Hierarchical
 - **Emerging:** Object-oriented, Object-relational
- Other classifications:
 - single-user or multi-user (most DBMSs)
 - centralized or distributed
 - Federated database (aka Multidatabase)
 - general-purpose or special-purpose

25

25

Cost Considerations for DBMSs

- Cost Range: from free open-source systems to configurations costing millions of dollars.
 - Examples of free DBMSs: MySQL, PostgreSQL, etc.
- Commercial DBMSs offer additional specialized modules; e.g., time-series module, spatial data module, document module, XML module.
 - These additional specialized modules are sometimes called **cartridges** (e.g., in Oracle).
- Different licensing options: site license, maximum number of concurrent users (seat license), single user, etc.

26

26